# Gap-Aware Retrieval (GAR)

A Gap-Aware Retrieval Algorithm for Iterative Retrieval-Augmented Generation

**Abstract**

Retrieval-Augmented Generation (RAG) systems usually select documents according to their relevance to a query. This can be insufficient for complex, multi-step questions because a document can be highly related to the topic while still failing to provide the information that is currently missing. This project proposes **Gap-Aware Retrieval (GAR)**, a retrieval algorithm that explicitly considers the current information gap and previously collected evidence when selecting documents.

MIGRES is used as the base system. MIGRES is a Missing Information Guided Retrieve-Extraction-Solving framework in which missing information is used to guide later retrieval. Its reported baseline uses BM25. GAR therefore replaces the document retrieval component while the remaining MIGRES pipeline is kept fixed as far as possible. The project compares MIGRES+GAR against MIGRES+BM25, DPR, Contriever, ColBERTv2, and SPLADE.

GAR is formulated using three proposed signals: gap relevance, gap coverage, and evidence novelty. The project evaluates retrieval quality, downstream answer quality, efficiency, ablations, and statistical significance. The main datasets are the six datasets reported in the MIGRES evaluation: WikiHop, HotpotQA, MuSiQue, Natural Questions, TriviaQA, and StrategyQA. The current formulation is a research proposal motivated by existing literature; a formal novelty claim requires a final literature and implementation check.

## 1 Introduction

Retrieval is a central component of modern Retrieval-Augmented Generation systems. A retriever is responsible for selecting a small set of documents from a large corpus so that a downstream model can use them to answer a question. A common approach is to rank documents according to their similarity to the query.

This approach is useful, but it has a weakness in iterative information-seeking tasks. After one retrieval step, the system may already know part of the answer. The next retrieval step should therefore not simply find documents that are relevant to the original question. It should find documents that provide the information that is still missing.

For example, consider the question "Why did Company X's revenue increase?" Suppose the system already knows that the revenue increased by 30 percent. A conventional retriever can repeatedly return documents mentioning the 30 percent increase. These documents are relevant to the topic, but they do not necessarily answer the remaining question: why did the increase happen?

This project studies whether retrieval can be improved by explicitly using the current information gap and the evidence already collected. The proposed algorithm is called Gap-Aware Retrieval (GAR).

## 2 About the Baseline Paper: MIGRES

MIGRES is the base system for this project. The MIGRES framework uses missing information to guide retrieval in an iterative process. Its Retrieval Module contains query generation, retrieval, and knowledge filtering components.

The important experimental idea is to change the retrieval component while leaving the rest of MIGRES fixed.

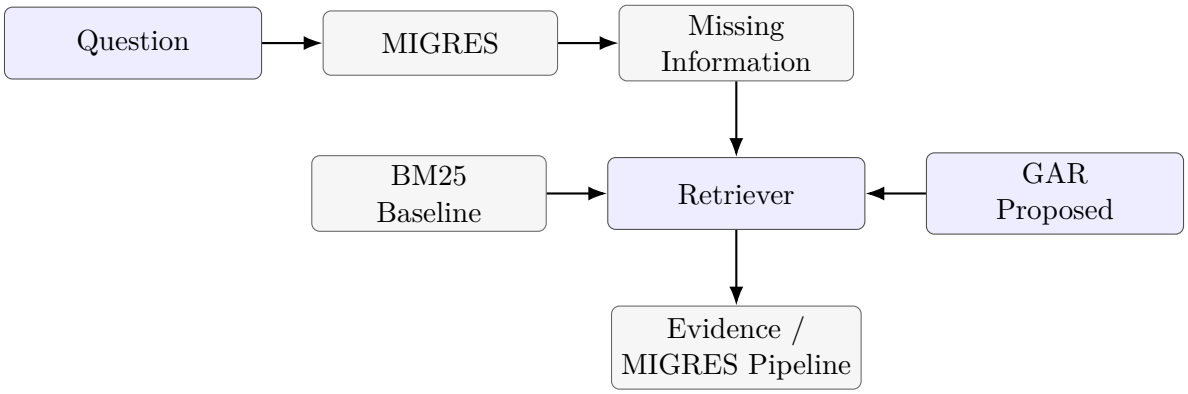



Figure 1: MIGRES is the fixed experimental framework; the retriever is the main experimental variable.
The original system is:

$$\text{MIGRES} + \text{BM25}.$$

Our proposed system is:

$$\text{MIGRES} + \text{GAR}.$$

The iterative loop remains part of MIGRES:

$$\text{Question} \rightarrow \text{Retrieval} \rightarrow \text{Evidence} \rightarrow \text{Missing Information} \rightarrow \text{Retrieval}.$$

GAR is the retrieval method used at the retrieval stage.

# 3 What Is an Information Retrieval Algorithm?

Information Retrieval (IR) is the task of finding useful information from a large collection of information. In our project, the collection is a document corpus and the input is a question or query. A simple retrieval system can be written as:

$$q + D \longrightarrow d_1, d_2, \ldots, d_k$$

where $q$ is the query, $D$ is the document collection, and the output is a ranked list of the best documents.
The important part is the scoring function. A retriever gives each candidate document a score and returns the documents with the highest scores. Different retrieval algorithms use different ways to calculate that score.

## 3.1 BM25

BM25 is a traditional sparse retrieval method. It mainly uses word matching between the query and document. It also considers how often a word appears, how common the word is in the whole collection, and document length. BM25 is our main baseline because MIGRES uses it in its reported retrieval setup.

## 3.2 DPR

Dense Passage Retrieval (DPR) represents questions and passages as dense vectors. It then compares the vectors to find semantically related passages . DPR is a strong dense retrieval baseline.

### 3.3 Contriever

Contriever is an unsupervised dense retrieval method. It learns useful representations without the same supervised retrieval training setup used by DPR . It gives us a different dense-retrieval baseline.

### 3.4 ColBERTv2

ColBERTv2 uses late interaction. Instead of representing the whole document and question using only one vector, it keeps token-level representations and compares them efficiently. This makes it a useful comparison with both sparse and single-vector dense retrieval .

### 3.5 SPLADE

SPLADE is a neural sparse retrieval method. It creates sparse representations while using a neural model to expand and weight useful terms. It combines some benefits of neural representation learning with efficient sparse retrieval .
Therefore, the main retrieval methods in our experiment are:

$$\boxed{\text{BM25, DPR, Contriever, ColBERTv2, SPLADE, GAR}}$$

GAR is different from the other methods because its score explicitly uses the current information gap and already collected evidence.

## 4 How We Made the GAR Equation

The equation is not copied from one paper. It is a simple combination of three ideas that already exist in the literature. We use those ideas as building blocks and then define one retrieval score for this project.
First, MIGRES gives us the idea of an **information gap**. MIGRES asks what information is still missing after the system has already collected some evidence . We use this idea through the variable $G$.
Second, Yadav, Bethard, and Surdeanu proposed an iterative evidence retrieval method that focuses later retrieval on terms that are not already covered by the current evidence. Their paper also defines a coverage measure . We use the general idea of **coverage of what is still missing**. We do not copy their exact formula.
Third, the Relevant Information Gain paper argues that retrieved information should be relevant but should also avoid unnecessary repetition. It proposes an information-gain objective so that useful diversity can appear among retrieved passages . We use this idea through the **novelty** term. Again, our equation is not the same as the equation in that paper.
Therefore, the three ideas become:

| Source | Idea used | GAR term |
|---|---|---|
| MIGRES | What information is missing? | $G$ |
| Yadav et al. | How much missing information is covered? | $C(d, G)$ |
| Relevant Information Gain | How much useful new information is added? | $N(d, E)$ |

We then keep the normal question-document relevance idea and combine all three signals into one score:

$$\boxed{S(d \mid q, E, G) = \alpha R(d, q, G) + \beta C(d, G) + \gamma N(d, E)}$$

In plain English, the equation says:

> **Choose documents that are relevant, that help fill the current gap, and that add useful information that we do not already have.**

This is the proposed GAR model. Its novelty should be stated carefully until the final literature review confirms that no previous paper uses the same complete formulation.

# 5 What Exactly Is GAR?

## 5.1 What GAR Is

GAR is a retrieval algorithm based on the idea that the usefulness of a document depends on the current state of the information need.
Instead of using only:

$$\mathrm{sim}(q, d),$$

GAR considers three questions:

1. Is the document relevant to the question and the current information gap?
2. Does the document cover information that is currently missing?
3. Does the document provide information that is not already present in the collected evidence?

The algorithm therefore attempts to distinguish between *relevant information* and *useful new information*.

## 5.2 How GAR Works

The input to GAR is:

$$(q, E, G, D),$$

where $q$ is the original question, $E$ is previously collected evidence, $G$ is the current information gap, and $D$ is the document corpus.
GAR scores candidate documents. The highest-scoring documents are returned.

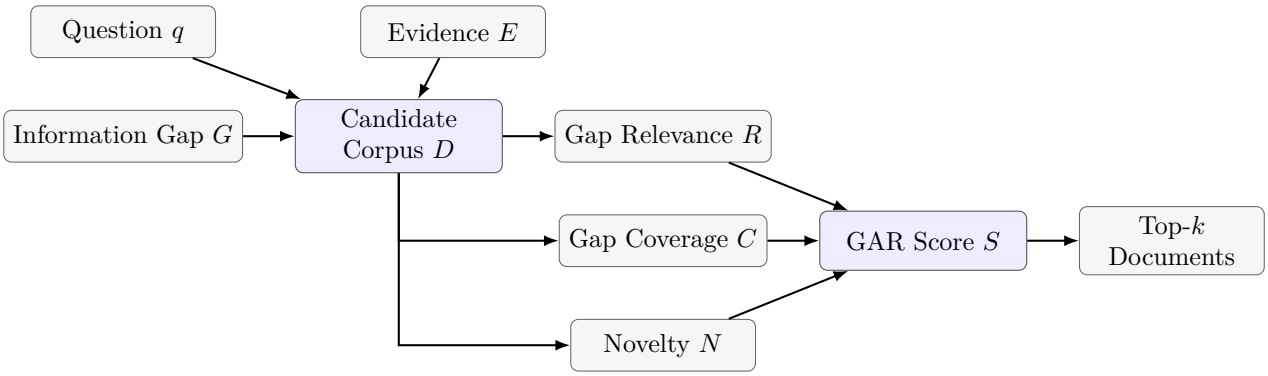



Figure 2: Conceptual architecture of the proposed Gap-Aware Retrieval algorithm.

## 5.3 Why It Can Be Useful

GAR is designed for settings where the system's information need changes after evidence is collected. Potential applications include multi-hop question answering, research-oriented RAG, evidence retrieval, OSINT-style information gathering, and other iterative information-seeking systems.
GAR does not require a separate agent or a new large language model. The surrounding system may supply the information gap and evidence state. GAR then performs document selection.

### 5.4 Advantages

The intended advantages are that GAR is modular, interpretable, and compatible with existing retrieval pipelines. Its score exposes why a document was preferred: relevance, coverage, and novelty. Another advantage is that it can be evaluated without creating a new benchmark. Existing datasets with supporting evidence can provide retrieval ground truth.

### 5.5 Disadvantages and Risks

The main risk is that novelty can reward documents that are different but irrelevant. Therefore novelty must not be used by itself.
Another challenge is operationalizing gap coverage without making the system too expensive. The project should avoid requiring a separate LLM call for every candidate document if a cheaper representation or matching mechanism can be used.
The weighting parameters can also change performance. They therefore need to be selected using validation data or learned in a controlled way.
Finally, if GAR uses BM25 to retrieve a candidate pool and only reranks that pool, it becomes a reranker rather than a complete first-stage retriever. The implementation must clearly define whether GAR itself retrieves from the corpus or only reranks externally generated candidates.

## 6 Mathematical Formulation

Let:

$$q = \text{original question}, \tag{1}$$
$$E = \text{evidence already collected}, \tag{2}$$
$$G = \text{current information gap}, \tag{3}$$
$$d = \text{candidate document}, \tag{4}$$
$$D = \text{document corpus}. \tag{5}$$

The proposed GAR score is:

$$\boxed{S(d \mid q, E, G) = \alpha R(d, q, G) + \beta C(d, G) + \gamma N(d, E)} \tag{1}$$

The retriever returns:

$$\boxed{TopK = \mathrm{TopK}_{d \in D}\, S(d \mid q, E, G)} \tag{2}$$

Equation (1) contains three terms: gap relevance, gap coverage, and evidence novelty.

### 6.1 Gap Relevance

Gap relevance is:

$$\boxed{R(d, q, G) = \lambda \,\mathrm{sim}(d, q) + (1 - \lambda)\,\mathrm{sim}(d, G)} \tag{3}$$

The first similarity measures how related the document is to the original question. The second measures how related it is to the information gap.
The parameter $\lambda$ controls the balance.
If $\lambda$ is high, the original question receives more weight. If $\lambda$ is low, the information gap receives more weight.
Similarity can be implemented using cosine similarity:

$$\boxed{\text{sim}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^T \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}} \tag{4}$$

For example, a document explaining why revenue increased should have higher similarity to a gap such as "cause of revenue increase" than a document that only discusses the company's stock price.

### 6.2 Gap Coverage

The second component is:

$$C(d, G).$$

The information gap can contain several requirements:

$$\boxed{G = \{g_1, g_2, \ldots, g_k\}} \tag{5}$$

A proposed weighted coverage score is:

$$\boxed{C(d, G) = \frac{\sum_{i=1}^{k} w_i c(d, g_i)}{\sum_{i=1}^{k} w_i}} \tag{6}$$

Here, $c(d, g_i)$ measures how well document $d$ covers requirement $g_i$, while $w_i$ represents the importance of that requirement.
For example, suppose:

$$G = \{\text{cause of revenue increase}, \text{supporting evidence}\}.$$

A document that only repeats the revenue number has low coverage. A document that explains the cause and provides evidence has higher coverage.
This coverage formulation is a proposed component of GAR. It should be treated as part of our research contribution rather than as a formula directly copied from MIGRES.

### 6.3 Evidence Novelty

The third component is:

$$N(d, E).$$

A simple proposed formulation is:

$$\boxed{N(d, E) = 1 - \max_{e \in E} \text{sim}(d, e)} \tag{7}$$

The purpose is to reduce the score of documents that repeat information already collected.
If a candidate is nearly identical to an existing evidence item:

$$\text{sim}(d, e) \approx 1,$$

then:

$$N(d, E) \approx 0.$$

If it is substantially different:

$$\text{sim}(d, e) \approx 0,$$

then:

$$N(d, E) \approx 1.$$

For the first retrieval step, where $E = \emptyset$, we use the neutral value

$$N(d, \emptyset) = 1.$$

This reflects the fact that no previous evidence exists against which the candidate document can be considered redundant.

### 6.4 Meaning of the Greek Letters

The Greek letters are weights:

$$\alpha = \text{weight of gap relevance},$$

$$\beta = \text{weight of gap coverage},$$

$$\gamma = \text{weight of evidence novelty}.$$

A convenient constraint is:

$$\boxed{\alpha + \beta + \gamma = 1, \qquad \alpha, \beta, \gamma \geq 0.} \tag{8}$$

The parameter $\lambda$ in Equation (3) is also a weight:

$$0 \leq \lambda \leq 1.$$

It determines how much importance is given to the original question versus the information gap. These parameters should be determined using validation experiments rather than chosen solely by intuition.

### 6.5 Numerical Example

Suppose a candidate document receives:

$$R = 0.80, \qquad C = 0.90, \qquad N = 0.60.$$

Let:

$$\alpha = 0.30, \qquad \beta = 0.50, \qquad \gamma = 0.20.$$

Then:

$$\begin{aligned} S(d) &= 0.30(0.80) + 0.50(0.90) + 0.20(0.60) \\ &= 0.24 + 0.45 + 0.12 \\ &= 0.81. \end{aligned}$$

Thus:

$$\boxed{S(d) = 0.81}.$$

The same calculation is performed for candidate documents. The highest-scoring candidates are returned.

# 7 MIGRES System for Our Project

MIGRES is the base system for this project. The MIGRES framework uses missing information to guide retrieval in an iterative process. Its Retrieval Module contains query generation, retrieval, and knowledge filtering components.
The important experimental idea is to change the retrieval component while leaving the rest of MIGRES fixed.

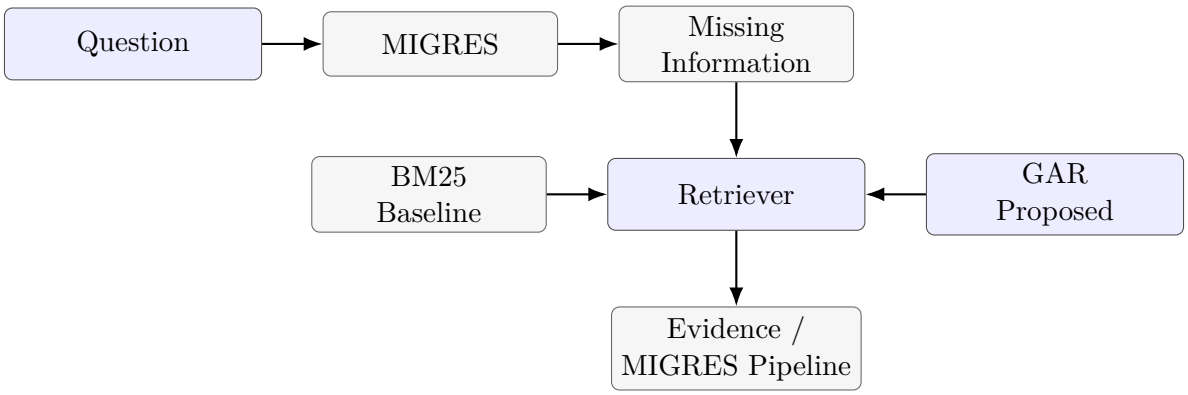



Figure 3: MIGRES is the fixed experimental framework; the retriever is the main experimental variable.
The original system is:

$$\text{MIGRES} + \text{BM25}.$$

Our proposed system is:

$$\text{MIGRES} + \text{GAR}.$$

The iterative loop remains part of MIGRES:

$$\text{Question} \rightarrow \text{Retrieval} \rightarrow \text{Evidence} \rightarrow \text{Missing Information} \rightarrow \text{Retrieval}.$$

GAR is the retrieval method used at the retrieval stage.

# 8 How the Datasets Are Used

We do not need to collect a new dataset for the first version of this project. The six datasets used by MIGRES are already public benchmarks with published papers. They provide questions, evidence, answers, or supporting annotations that can be used for evaluation.
The dataset is used in two different ways. During retrieval, GAR sees only the information that a real system is allowed to see: the question, current evidence, information gap, and document corpus. During evaluation, the benchmark's supporting evidence or answer annotations are used as ground truth. GAR must not see the ground-truth evidence while retrieving.
For development, HotpotQA FullWiki is a sensible first dataset because it contains supporting facts and a large Wikipedia retrieval setting. After the implementation works, the remaining MIGRES datasets can be used to test whether the method generalizes.

# 9 Datasets

## 9.1 Dataset Selection

MIGRES's main evaluation reports six datasets:

1. WikiHop,
2. HotpotQA,
3. MuSiQue,
4. Natural Questions (NQ),
5. TriviaQA,
6. StrategyQA.

These datasets cover different forms of open-domain, multi-hop, and reasoning-oriented question answering.
The initial development should use HotpotQA FullWiki because it provides explicit supporting facts and a full-corpus retrieval setting. After the implementation is stable, the other MIGRES datasets can be used for generalization.
No new dataset needs to be collected for the initial project.

### 9.2 WikiHop

WikiHop is a multi-hop reading-comprehension dataset constructed to require information to be combined across documents. It contains approximately 51K examples in the published benchmark. Its relevance to GAR is that the system must identify useful documents from candidate evidence rather than relying on one isolated passage.
Dataset paper: https://aclanthology.org/Q18-1021/

### 9.3 HotpotQA

HotpotQA contains approximately 113K Wikipedia-based question-answer pairs. It is particularly useful for retrieval research because it contains sentence-level supporting facts and multi-hop questions. Two commonly used settings are the distractor setting and the FullWiki setting. FullWiki is more appropriate for a first-stage retrieval experiment because the retriever searches the larger processed Wikipedia corpus.
The official HotpotQA resources provide data and a processed Wikipedia corpus, and the repository supports replacing retrieval with a custom IR system.
Dataset paper: https://aclanthology.org/D18-1259/
Official resources: https://hotpotqa.github.io/

### 9.4 MuSiQue

MuSiQue contains approximately 25K questions designed around compositional multi-hop reasoning. The benchmark focuses on connecting multiple reasoning steps instead of simply retrieving unrelated evidence.
This is relevant to GAR because a retrieval algorithm should be able to support different information needs across multiple steps.
Dataset paper: https://aclanthology.org/2022.tacl-1.31/
Official repository: https://github.com/StonyBrookNLP/musique

### 9.5 Natural Questions

Natural Questions is based on real, anonymized, aggregated questions issued to Google Search. Annotators were shown Wikipedia pages and identified long answers and short answers where applicable.
The released dataset contains 307,373 training examples, 7,830 development examples, and 7,842 test examples.

NQ is useful because it is not designed exclusively as a multi-hop benchmark. It therefore helps test whether the gap-aware idea remains useful in a broader open-domain setting.
Dataset paper: https://aclanthology.org/Q19-1026/
Official data: https://ai.google.com/research/NaturalQuestions/download

### 9.6 TriviaQA

TriviaQA contains approximately 95K question-answer pairs and more than 650K question-answer-evidence triples in its published release.
Questions were authored by trivia enthusiasts, and evidence documents were gathered independently. This makes it useful for studying retrieval from heterogeneous evidence.
Dataset paper: https://aclanthology.org/P17-1147/
Official resources: https://nlp.cs.washington.edu/triviaqa/

### 9.7 StrategyQA

StrategyQA contains 2,780 verified questions. It focuses on implicit multi-step reasoning. The dataset includes reasoning decompositions and evidence paragraphs associated with reasoning steps.
This makes StrategyQA useful for testing retrieval when the required reasoning path is not explicitly written in the question.
Dataset paper: https://aclanthology.org/2021.tacl-1.21/
Official resources: https://allenai.org/data/strategyqa

### 9.8 Dataset Summary

| Dataset | Approx. scale | Important feature |
|---|---|---|
| WikiHop | 51K examples | Multi-document, multi-hop reasoning. |
| HotpotQA | 113K QA pairs | Multi-hop questions and sentence-level supporting facts. |
| MuSiQue | 25K questions | Compositional 2–4 hop reasoning. |
| NQ | 307,373 train; 7,830 dev; 7,842 test | Real search questions with Wikipedia evidence. |
| TriviaQA | 95K QA pairs; >650K evidence triples | Heterogeneous evidence and open-domain QA. |
| StrategyQA | 2,780 verified questions | Implicit reasoning with evidence for reasoning steps. |

Table 1: Published dataset characteristics. Exact numbers can vary by release, preprocessing, and split. The exact files loaded in the experiment must be taken from the MIGRES repository configuration.

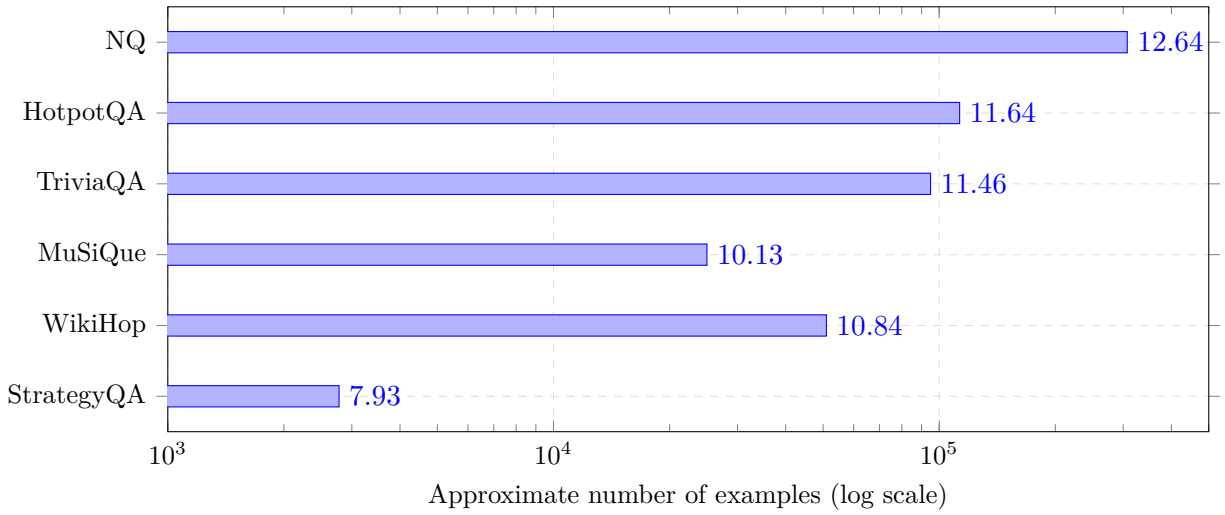



Figure 4: Approximate published dataset sizes. The logarithmic axis is used because the dataset sizes differ substantially.

## 10 How We Will Use GAR with MIGRES

The implementation will be controlled. First, we reproduce the original MIGRES system with BM25. This gives us a working baseline. Next, we find the exact retrieval interface in the MIGRES code. GAR is then implemented with the same input-output role as the original retriever.
The main comparison is:

$$\boxed{\text{MIGRES+BM25} \quad \textit{vs.} \quad \text{MIGRES+GAR}}$$

After that, we add the other retrieval baselines:

$$\text{MIGRES+BM25}, \quad \text{MIGRES+DPR}, \quad \text{MIGRES+Contriever},$$
$$\text{MIGRES+ColBERTv2}, \quad \text{MIGRES+SPLADE}, \quad \text{MIGRES+GAR}.$$

The same dataset, corpus, questions, and MIGRES settings should be used as far as possible. This makes the retrieval method the main changing variable.
A very important implementation point is the difference between a retriever and a reranker. A complete GAR retriever should search the corpus itself:

$$(q, E, G, D) \rightarrow GAR \rightarrow TopK.$$

If GAR first receives a BM25 candidate list and only reorders those documents, then it is a reranker. That is a different research claim. Because this project is about retrieval, we should aim for the first design.

## 11 Experimental and Evaluation Plan

The experiment should be controlled so that the retrieval algorithm is the main variable.
The main idea is simple: we keep the MIGRES system, datasets, question sets, and evaluation procedure as similar as possible across all experiments. We change the retrieval method and measure how the change affects retrieval quality, final answer quality, efficiency, and cost.
The systems are:

| System | Retriever | Implementation Status |
|---|---|---|
| MIGRES + BM25 | BM25 | Existing implementation |
| MIGRES + DPR | DPR | Existing implementation and pretrained models |
| MIGRES + Contriever | Contriever | Existing implementation and pretrained models |
| MIGRES + ColBERTv2 | ColBERTv2 | Existing implementation and pretrained models |
| MIGRES + SPLADE | SPLADE | Existing implementation and pretrained models |
| MIGRES + GAR | GAR | Proposed and implemented by us |

### 11.1 Why We Do Not Implement the Baselines From Scratch

The five comparison algorithms are established information retrieval methods. Each has a published mathematical formulation and an existing implementation, and several have publicly available pretrained models.
Therefore, we do not need to reimplement their complete retrieval algorithms from the mathematical definitions. Doing so could introduce implementation differences that make the comparison less fair. Instead, we use their existing implementations or established retrieval libraries and connect them to the same experimental framework.
Our contribution is the implementation of GAR.
The experimental structure is therefore:

$$\boxed{\text{Existing Retrieval Algorithms} \rightarrow \text{Use Their Implementations}}$$

while:

$$\boxed{\text{GAR} \rightarrow \text{Implement Our Proposed Model}}$$

This makes the experiment more practical and also makes the comparison easier to reproduce.

### 11.2 BM25

BM25 is a traditional sparse retrieval algorithm. It has a well-defined mathematical scoring function based mainly on query-term frequency, inverse document frequency, and document-length normalization.
A simplified form is:

$$BM25(q,d) = \sum_{t \in q} IDF(t) \frac{f(t,d)(k_1+1)}{f(t,d) + k_1 \left(1 - b + b\frac{|d|}{avgdl}\right)}.$$

Here, $f(t,d)$ is the frequency of term $t$ in document $d$, $|d|$ is the document length, $avgdl$ is the average document length, and $k_1$ and $b$ are parameters.
For our project, BM25 is especially important because it is the main retrieval baseline used by MIGRES.
We can use an existing implementation rather than writing BM25 from scratch. Options include the Python package `rank_bm25` and the Pyserini toolkit. Pyserini provides reproducible first-stage retrieval with sparse methods such as BM25 and also supports dense retrieval .
Official resources:
https://github.com/dorianbrown/rank_bm25
https://github.com/castorini/pyserini

### 11.3 DPR

Dense Passage Retrieval (DPR) uses a dense representation of the question and a dense representation of each passage.
A simplified retrieval score is:

$$S_{DPR}(q, d) = \mathbf{q}^T \mathbf{d},$$

where $\mathbf{q}$ is the question vector and $\mathbf{d}$ is the document vector.
The original DPR paper provides the mathematical model and describes the dual-encoder architecture .
An official Facebook Research implementation is available. It contains code for generating document embeddings, generating query embeddings, performing retrieval, and evaluating retrieval results. The repository also provides pretrained checkpoints .
Therefore, we do not need to train DPR from scratch for the initial comparison.
Official implementation:
https://github.com/facebookresearch/DPR
The implementation can generate document embeddings, retrieve documents using those embeddings, and evaluate top-$k$ retrieval results.

### 11.4 Contriever

Contriever is an unsupervised dense retrieval method. It represents queries and documents as dense vectors and retrieves documents according to their vector similarity.
Its basic retrieval operation can be represented as:

$$S_{Contriever}(q, d) = sim(\mathbf{q}, \mathbf{d}).$$

The original Contriever work provides the retrieval method and pretrained models .
The official Facebook Research repository contains preprocessing and retrieval code, and pretrained Contriever models are available for use .
Therefore, we can use a pretrained Contriever model and generate document and query representations rather than training a new Contriever model.
Official implementation:
https://github.com/facebookresearch/contriever
This gives us a second dense-retrieval baseline that is different from DPR because Contriever is trained using an unsupervised contrastive-learning approach.

### 11.5 ColBERTv2

ColBERTv2 uses a late-interaction retrieval architecture.
Instead of representing an entire document using only one vector, ColBERT keeps token-level representations.
A simplified form of its score is:

$$S_{ColBERT}(q, d) = \sum_{i \in q} \max_{j \in d} sim(\mathbf{q}_i, \mathbf{d}_j).$$

Here, $\mathbf{q}_i$ is the representation of a query token and $\mathbf{d}_j$ is the representation of a document token.
This allows the method to perform a more detailed comparison between query and document content.
ColBERTv2 has an official implementation and pretrained models. Therefore, we can use the existing implementation instead of implementing the late-interaction mechanism ourselves.
Official implementation:
https://github.com/stanford-futuredata/ColBERT
ColBERTv2 provides a useful comparison because its retrieval mechanism is substantially different from both BM25 and single-vector dense retrieval.

### 11.6 SPLADE

SPLADE is a neural sparse retrieval method.

Instead of representing a document only with a dense vector, SPLADE produces a sparse representation over a vocabulary. This representation can then be indexed using sparse retrieval structures.
A simplified representation is:

$$\mathbf{s}_d = f_\theta(d),$$

where $f_\theta$ is the SPLADE model and $\mathbf{s}_d$ is the sparse document representation.
The retrieval score can then be calculated using a sparse similarity operation such as a dot product:

$$S_{SPLADE}(q, d) = \mathbf{s}_q^T \mathbf{s}_d.$$

The original SPLADE work provides the mathematical formulation and training method .
There are existing implementations and pretrained models, so we do not need to train SPLADE from scratch.
The official SPLADE repository provides the original implementation:
https://github.com/naver/splade
There are also practical indexing implementations such as `splade-index`, which can create and search SPLADE indexes using Python:
https://github.com/rasyosef/splade-index
This makes SPLADE practical to include as a baseline.

### 11.7 GAR

GAR is different from the five baselines because it is the method proposed in this project.
We will implement GAR ourselves using the proposed mathematical formulation:

$$\boxed{S(d \mid q, E, G) = \alpha R(d, q, G) + \beta C(d, G) + \gamma N(d, E)}$$

The five existing algorithms mainly ask:

"How relevant is document $d$ to query $q$?"

GAR additionally considers:

"What information is still missing?"

and:

"What information has already been collected?"

Therefore, GAR uses:

$$q, \quad G, \quad E, \quad d$$

instead of only the original query and candidate document.
The GAR implementation will calculate:

$$R(d, q, G)$$

for gap relevance,

$$C(d, G)$$

for gap coverage, and

$$N(d, E)$$

for evidence novelty.

The final score is:

$$S(d \mid q, E, G) = \alpha R(d, q, G) + \beta C(d, G) + \gamma N(d, E).$$

The highest-scoring documents are returned:

$$TopK = \mathrm{TopK}_{d \in D}\, S(d \mid q, E, G).$$

### 11.8 Implementation Summary

The practical implementation plan is therefore:

| **Method** | **What We Use** | **What We Build** |
|---|---|---|
| BM25 | Existing IR implementation | Integration only |
| DPR | Existing code + pretrained model | Integration only |
| Contriever | Existing code + pretrained model | Integration only |
| ColBERTv2 | Existing code + pretrained model | Integration only |
| SPLADE | Existing code + pretrained model | Integration only |
| GAR | Our mathematical model | **Full implementation** |

This keeps the project focused. We are not trying to reproduce five large retrieval systems from scratch. We are building one new retrieval method and comparing it with established methods.

### 11.9 Common Retrieval Interface

For a fair comparison, all retrieval methods should expose the same basic interface:

$$Retriever(q, E, G, D, k) \rightarrow (d_1, s_1), \ldots, (d_k, s_k).$$

For the existing retrieval methods, some of the inputs may not be used.
For example, BM25 mainly uses:

$$(q, D).$$

DPR mainly uses:

$$(q, D).$$

Contriever mainly uses:

$$(q, D).$$

ColBERTv2 mainly uses:

$$(q, D).$$

SPLADE mainly uses:

$$(q, D).$$

GAR uses:

$$(q, E, G, D).$$

This difference is intentional. GAR is designed to use the current information state.
The output should be standardized so that every method returns document identifiers and retrieval scores:

$$[(d_1, s_1), (d_2, s_2), \ldots, (d_k, s_k)].$$

This makes it possible to evaluate all methods using the same retrieval metrics.

### 11.10 Important Experimental Control

The comparison should keep the following factors fixed wherever possible:

- the same question set;
- the same document corpus;
- the same train, validation, and test splits;
- the same top-$k$ retrieval setting;
- the same MIGRES components outside the retriever;
- the same answer-generation model;
- the same evaluation metrics;
- the same stopping conditions for iterative retrieval.

Only the retrieval method should change.
This is important because otherwise an improvement could come from a different language model, corpus, number of retrieved documents, or prompt rather than from the retrieval algorithm.

### 11.11 Models, Packages, and Resources We Can Use

The project can use existing implementations and pretrained models for the baseline methods.
For BM25, `rank_bm25` is a simple Python option, while Pyserini is a stronger option when we need reproducible large-scale indexing and retrieval. Pyserini supports sparse retrieval such as BM25 as well as dense retrieval and provides a Python interface .
For DPR, the Facebook Research DPR repository provides the implementation, retrieval scripts, embedding generation, evaluation code, and pretrained checkpoints .
For Contriever, the Facebook Research repository provides the original implementation and model resources .
For ColBERTv2, the official ColBERT repository provides the implementation and model-based retrieval pipeline .
For SPLADE, the official SPLADE implementation and pretrained models can be used. A separate Python indexing implementation, `splade-index`, is also available for creating and searching sparse indexes .
For GAR, we implement the retrieval score ourselves.
The project therefore follows this principle:

| Use established implementations for baselines; build GAR ourselves. |
|---|

### 11.12 Why This Is a Fair Comparison

The purpose of the experiment is not to determine which implementation is easiest to code. The purpose is to compare retrieval strategies.
Therefore, using established implementations for the baselines is preferable to creating simplified versions of those algorithms.
Each baseline should use a published or officially released implementation where possible. The exact model checkpoint, index type, embedding model, and retrieval parameters must be recorded in the final experiment report.
For reproducibility, we should create a configuration table containing:

| Parameter | Value to Record |
|---|---|
| Retriever | BM25 / DPR / Contriever / ColBERTv2 / SPLADE / GAR |
| Model checkpoint | Exact model name or path |
| Corpus | Exact corpus version |
| Top-$k$ | Exact value |
| Embedding model | Exact model if applicable |
| GAR $\alpha$ | Validation value |
| GAR $\beta$ | Validation value |
| GAR $\gamma$ | Validation value |
| GAR $\lambda$ | Validation value |
| Hardware | CPU/GPU configuration |
| Software versions | Python and package versions |
| MIGRES configuration | Exact configuration used |

This prevents accidental differences between experiments.

### 11.13 Computational and API Cost Comparison

The experiment should also measure the computational cost of each method.
For each system, we should record retrieval latency and, where applicable, indexing time and memory usage.
For the complete MIGRES system, we should also record:

$$\text{LLM calls per question},$$

$$\text{input tokens},$$

$$\text{output tokens},$$

and:

$$\text{API cost per question}.$$

The important point is that the baseline retrievers themselves do not necessarily require an LLM API call for every candidate document. DPR, Contriever, ColBERTv2, and SPLADE can use locally loaded pretrained models and locally calculated representations.
GAR should follow the same principle.
The proposed GAR score:

$$S(d \mid q, E, G) = \alpha R(d, q, G) + \beta C(d, G) + \gamma N(d, E)$$

should preferably be calculated using local representations or other low-cost operations rather than making a separate LLM API call for every candidate document.
Therefore, one of our practical goals is:

$$\boxed{\text{Improve retrieval without adding unnecessary LLM API calls.}}$$

The final comparison should report both quality and cost rather than reporting retrieval accuracy alone.

## 11.14 Final Experimental Comparison

The final experiment therefore has six systems:

| | |
|---|---|
| MIGRES+BM25 | Traditional sparse retrieval |
| MIGRES+DPR | Dense bi-encoder retrieval |
| MIGRES+Contriever | Unsupervised dense retrieval |
| MIGRES+ColBERTv2 | Late-interaction retrieval |
| MIGRES+SPLADE | Neural sparse retrieval |
| MIGRES+GAR | **Proposed gap-aware retrieval** |

The main comparison is not simply:

Which algorithm has the highest score?

It is:

Does explicitly modeling the information gap improve retrieval?

We therefore evaluate retrieval quality, final MIGRES answer quality, ablation results, latency, LLM calls, token usage, and API cost.
The same dataset, corpus, question set, MIGRES modules, stopping conditions, and evaluation procedure should be used wherever possible.

## 11.15 Retrieval Metrics

**Recall@k** measures whether required evidence appears in the top-$k$ retrieved documents.

$$Recall@k = \frac{\text{relevant evidence retrieved in top } k}{\text{total relevant evidence}}.$$

**Precision@k** measures how much of the top-$k$ retrieval is relevant:

$$Precision@k = \frac{\text{relevant documents in top } k}{k}.$$

**MRR** measures how early the first relevant document appears:

$$MRR = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{rank_i}.$$

**nDCG** can be used when relevance is graded rather than binary.
Because GAR is specifically gap-aware, we should also report an evidence-coverage measure where the dataset's annotations make this possible.

## 11.16 Downstream Metrics

The retrieval results should also be evaluated through the downstream MIGRES task. Depending on the dataset and available annotations, this can include Exact Match, F1, answer accuracy, and evidence/citation quality.
Efficiency should also be measured, including latency, number of retrieval iterations, external-knowledge tokens, and LLM/API calls where applicable.

## 11.17 API Cost and LLM Call Efficiency

An important practical goal of GAR is to improve retrieval without requiring many additional calls to a large language model (LLM).

The GAR equation itself does not require an LLM call for every candidate document. This is important for keeping the system practical. The similarity, coverage, and novelty calculations can be implemented using local embeddings, vector similarity, lexical matching, or other low-cost operations. The intended flow is:

$$\text{Question} \rightarrow \text{MIGRES identifies information gap} \rightarrow \text{GAR retrieves documents}$$

GAR should not call an LLM separately for every candidate document. For example, if a corpus search produces 1,000 candidate documents, sending all 1,000 documents to an LLM one by one would be expensive and slow. Instead, GAR should calculate its retrieval score using precomputed or locally calculated representations.
The proposed score remains:

$$S(d \mid q, E, G) = \alpha R(d, q, G) + \beta C(d, G) + \gamma N(d, E).$$

If the required representations are generated locally, these calculations do not require an additional LLM API call for every candidate document.
This gives us an important practical research question:

$$\boxed{\text{Can GAR improve retrieval quality without increasing LLM API usage?}}$$

We therefore measure:

$$\text{Number of LLM Calls per Question}$$

$$\text{Input Tokens per Question}$$

$$\text{Output Tokens per Question}$$

$$\text{Total Tokens per Question}$$

and, when the API provider's pricing is known:

$$\boxed{\text{API Cost} = \text{Input Cost} + \text{Output Cost}} \tag{11}$$

If the input and output prices are $p_{in}$ and $p_{out}$ per token, then:

$$\boxed{Cost(q) = T_{in}(q)p_{in} + T_{out}(q)p_{out}} \tag{12}$$

where:

$$T_{in}(q) = \text{input tokens used for question } q$$

and:

$$T_{out}(q) = \text{output tokens generated for question } q.$$

The average cost over $N$ questions is:

$$\boxed{Average\ Cost = \frac{1}{N}\sum_{i=1}^{N} Cost(q_i)} \tag{13}$$

We compare the complete systems, not only the retrieval component:

$$\text{MIGRES+BM25}$$

versus:

$$\text{MIGRES+GAR.}$$

The same measurements can also be collected for DPR, Contriever, ColBERTv2, and SPLADE. Retrieval quality and cost should be studied together. A method that gives a small accuracy improvement but requires many additional LLM calls may not be a useful practical improvement. We therefore investigate whether GAR can maintain or improve retrieval quality without increasing LLM calls or API cost. Any reduction in calls, tokens, or cost is treated as an empirical result rather than an assumption.

### 11.18 Research Questions

The main research questions are:

1. **RQ1:** Does GAR retrieve more useful evidence than BM25?
2. **RQ2:** Does GAR outperform strong dense, sparse, and late-interaction retrieval baselines?
3. **RQ3:** Does replacing BM25 with GAR improve MIGRES's downstream performance?
4. **RQ4:** Which GAR component contributes most to retrieval quality?
5. **RQ5:** Can GAR improve retrieval quality without increasing the number of LLM API calls?
6. **RQ6:** Can GAR reduce token usage or API cost while maintaining or improving answer quality?
7. **RQ7:** Does GAR generalize across the six datasets used by MIGRES?

### 11.19 Hypothesis Testing

The primary null hypothesis is:

$$H_0 : \text{GAR does not improve retrieval performance over BM25.}$$

The alternative hypothesis is:

$$H_1 : \text{GAR improves retrieval performance over BM25.}$$

A secondary comparison can test GAR against the strongest baseline:

$$H_0 : GAR \leq \text{Best Baseline}$$

versus:

$$H_1 : GAR > \text{Best Baseline.}$$

The exact statistical test should be selected after examining the per-question metric distributions. The final report should include effect sizes and confidence intervals as well as statistical significance.

### 11.20 API Cost Hypotheses

The API-cost analysis has secondary hypotheses.
For the number of LLM calls:

$$H_0 : \text{MIGRES+GAR uses the same or more LLM calls than MIGRES+BM25.}$$

$$H_1 : \text{MIGRES+GAR uses fewer LLM calls than MIGRES+BM25.}$$

For token usage:

$$H_0 : \text{MIGRES+GAR does not reduce token usage.}$$

$$H_1 : \text{MIGRES+GAR reduces token usage.}$$

For API cost:

$$H_0 : \text{MIGRES+GAR does not reduce API cost.}$$

$$H_1 : \text{MIGRES+GAR reduces API cost.}$$

These are secondary hypotheses. The primary hypothesis of the project remains whether GAR improves retrieval quality over the baseline.

### 11.21 Ablation Studies

Ablations are necessary because a better score from the complete GAR model would not by itself show that all three terms are useful.
We therefore evaluate:

$$GAR_{-R}$$

without gap relevance,

$$GAR_{-C}$$

without gap coverage, and

$$GAR_{-N}$$

without evidence novelty.
The full model is the weighted combination

$$GAR = \alpha R + \beta C + \gamma N.$$

The ablations remove one corresponding weighted component:

$$GAR_{-R} = \beta C + \gamma N, \qquad GAR_{-C} = \alpha R + \gamma N, \qquad GAR_{-N} = \alpha R + \beta C.$$

The results can show whether gap awareness, coverage, and novelty each contribute to performance.

## 12 Conclusion

This project proposes Gap-Aware Retrieval, a retrieval algorithm that changes the retrieval objective from simple query relevance toward information-gap-aware evidence selection.
MIGRES supplies the experimental setting because it explicitly identifies missing information during iterative retrieval. GAR then replaces the BM25 retrieval component.
The central mathematical formulation is:

$$\boxed{S(d \mid q, E, G) = \alpha R(d, q, G) + \beta C(d, G) + \gamma N(d, E)}$$

The project compares:

$$\text{MIGRES+BM25}, \quad \text{MIGRES+DPR}, \quad \text{MIGRES+Contriever},$$

MIGRES+ColBERTv2, MIGRES+SPLADE, MIGRES+GAR.

The evaluation uses the six datasets reported by MIGRES and includes retrieval metrics, downstream answer quality, ablations, latency, LLM call counts, token usage, API cost, and hypothesis testing. The key research question is:

> **Can explicitly modeling the information gap and previously collected evidence produce more useful retrieval than standard retrieval methods?**

## 13 References


1. Wang, K., Duan, F., Li, P., Wang, S., and Cai, X. (2025). *LLMs Know What They Need: Leveraging a Missing Information Guided Framework to Empower Retrieval-Augmented Generation.* COLING 2025. https://aclanthology.org/2025.coling-main.163/

2. Feng, X. et al. (2020). *Unsupervised Alignment-based Iterative Evidence Retrieval.* ACL 2020. https://aclanthology.org/2020.acl-main.414/

3. Asai, A. et al. (2019). *Multi-Hop Paragraph Retrieval.* ACL 2019. https://aclanthology.org/P19-1222/

4. *Better RAG using Relevant Information Gain.* (2024). https://arxiv.org/abs/2407.12101

5. *InfoGain-RAG.* (2025). EMNLP 2025. https://aclanthology.org/2025.emnlp-main.365/

6. Karpukhin, V. et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering.* EMNLP 2020. https://aclanthology.org/2020.emnlp-main.550/

7. Santhanam, K. et al. (2022). *ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction.* NAACL 2022. https://aclanthology.org/2022.naacl-main.272/

8. Izacard, G. et al. (2022). *Unsupervised Dense Information Retrieval with Contrastive Learning.* EMNLP 2022. https://aclanthology.org/2022.emnlp-main.738/

9. Formal, T., Lassance, C., Piwowarski, B., and Clinchant, S. (2021). *SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking.* arXiv. https://arxiv.org/abs/2107.05720

10. Asai, A. et al. (2022). *TART: Task-aware Retrieval with Instructions.* arXiv. https://arxiv.org/abs/2211.09260

11. Wang, L. et al. (2023). *Query2doc: Query Expansion with Large Language Models.* EMNLP 2023. https://aclanthology.org/2023.emnlp-main.585/

12. Gao, L. et al. (2023). *Precise Zero-Shot Dense Retrieval without Relevance Labels.* ACL 2023. https://aclanthology.org/2023.acl-long.99/

13. *EfficientRAG.* (2024). EMNLP 2024. https://aclanthology.org/2024.emnlp-main.199/

14. *IterCQR.* (2024). NAACL 2024. https://aclanthology.org/2024.naacl-long.449/

15. Jeong, S. et al. (2024). *Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity.* NAACL 2024. https://aclanthology.org/2024.naacl-long.389/

## Dataset References

**WikiHop:** Welbl, J., Stenetorp, P., and Riedel, S. (2018). *Constructing Datasets for Multi-hop Reading Comprehension Across Documents*. TACL. https://aclanthology.org/Q18-1021/
**HotpotQA:** Yang, Z. et al. (2018). *HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering*. EMNLP. https://aclanthology.org/D18-1259/
**MuSiQue:** Trivedi, H. et al. (2022). *MuSiQue: Multihop Questions via Single-hop Question Composition*. TACL. https://aclanthology.org/2022.tacl-1.31/
**Natural Questions:** Kwiatkowski, T. et al. (2019). *Natural Questions: A Benchmark for Question Answering Research*. TACL. https://aclanthology.org/Q19-1026/
**TriviaQA:** Joshi, M., Choi, E., Weld, D., and Zettlemoyer, L. (2017). *TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension*. ACL. https://aclanthology.org/P17-1147/
**StrategyQA:** Geva, M. et al. (2021). *Did Aristotle Use a Laptop? A Question Answering Benchmark with Implicit Reasoning Strategies*. TACL. https://aclanthology.org/2021.tacl-1.21/

## Official Code and Data Resources

**MIGRES:** https://github.com/AdelWang/MIGRES
**HotpotQA:** https://github.com/hotpotqa/hotpot
**MuSiQue:** https://github.com/StonyBrookNLP/musique
**StrategyQA:** https://github.com/eladsegal/strategyqa
**Natural Questions:** https://ai.google.com/research/NaturalQuestions/download
**TriviaQA:** https://nlp.cs.washington.edu/triviaqa/

## Implementation Resources and Existing Retrieval Tools

The following existing implementations and software resources can be used to reproduce the baseline retrieval methods. These resources allow us to use established implementations rather than implementing the baseline algorithms from scratch.

- **DPR – Dense Passage Retrieval**
  Facebook AI Research. Official implementation. https://github.com/facebookresearch/DPR
- **Contriever**
  Facebook AI Research. Official implementation. https://github.com/facebookresearch/contriever
- **ColBERT**
  Stanford Future Data. Official implementation. https://github.com/stanford-futuredata/ColBERT
- **SPLADE**
  Naver Labs. Official implementation. https://github.com/naver/splade
- **SPLADE-Index**
  Python indexing implementation for SPLADE. https://github.com/rasyosef/splade-index
- **Pyserini**
  Lin, Y. et al. A Python toolkit for reproducible information retrieval research with sparse and dense representations. https://github.com/castorini/pyserini

- **rank_bm25**
  Brown, D. Python implementation of BM25 and related ranking functions. https://github.com/dorianbrown/rank_bm25